



Pokémon Matchup Predictor



ณัฐพงศ์ คำสียา 6710450813 หมู่ 200
วีรวิชญ์ นิธิศไพศาลกุล 6710451313 หมู่ 200

01418262 Machine Learning System 1/2568
มหาวิทยาลัยเกษตรศาสตร์
คณะวิทยาศาสตร์ ภาควิชาวิทยาการคอมพิวเตอร์



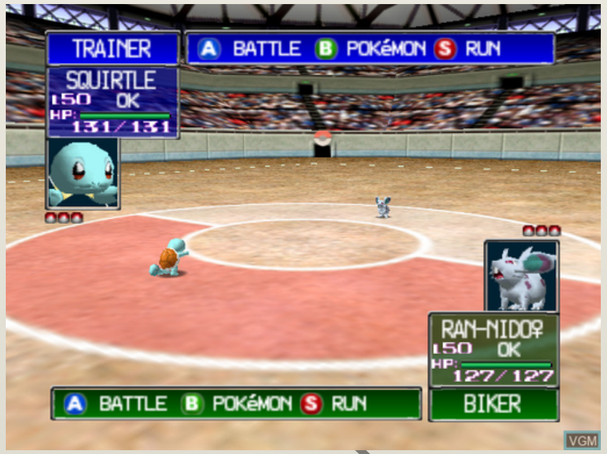


INTRODUCTION





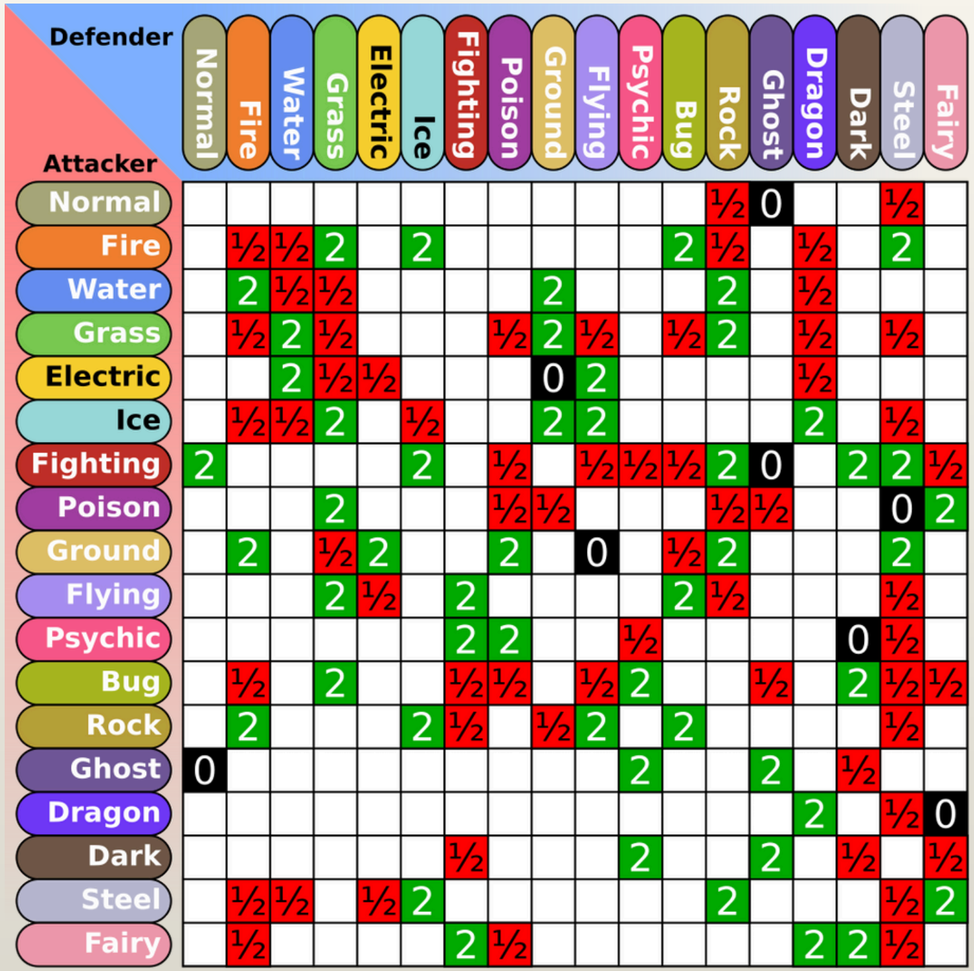




WHO WOULD WIN ?

ที่มาของระบบ

เนื่องจาก**ผู้เล่นใหม่**อาจจะมีปัญหาเกี่ยวกับ matchup ของโปเกมอนแต่ละตัวที่สู้กัน เพราะโปเกมอนแต่ละตัวมีค่าพลังและธาตุที่ไม่เหมือนกัน การที่ผู้เล่นใหม่จะคาดเดาว่าโปเกม่อนของตนสามารถสู้กับโปเกม่อนคู่ต่อสู้ได้หรือไม่ เป็นเรื่องที่ยุ่งยากและเสียเวลา เพราะหากไม่ชำนาญก็ต้องค้นหาข้อมูลจาก Wiki/Database เพื่อทราบธาตุของโปเกม่อนทั้ง 2 อีกทั้งยังต้องดู Pokémon Type Chart เพื่อดู **Effectiveness** ของธาตุนั้น ๆ พวกผมจึงคิดว่าการที่เราสามารถคำนวณเปอร์เซ็นต์การชนะ/แพ้ของ matchup ไปเลย จะทำให้การเล่นเกมโปเกม่อน**ง่ายขึ้น**ไม่มากก็น้อย



Attacker \ Defender	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Normal													½	0			½	
Fire		½	½	2		2						2	½		½		2	
Water		2	½	½					2				2		½			
Grass		½	2	½				½	2	½		½	2		½		½	
Electric			2	½	½				0	2					½			
Ice		½	½	2		½			2	2					2		½	
Fighting	2					2		½		½	½	½	2	0		2	2	½
Poison				2				½	½				½	½			0	2
Ground		2		½	2			2		0		½	2				2	
Flying				2	½		2					2	½				½	
Psychic							2	2			½					0	½	
Bug		½		2			½	½		½	2			½		2	½	½
Rock		2				2	½		½	2		2					½	
Ghost	0										2			2		½		
Dragon															2		½	0
Dark							½				2			2		½		½
Steel		½	½		½	2							2				½	2
Fairy		½					2	½							2	2	½	

วัตถุประสงค์ของระบบ

1

เพื่อความสะดวกสบายของผู้เล่น โดยลดขั้นตอนในการหาข้อมูลทำให้ผู้เล่นไม่ต้องสลับหน้าจอไป-มาเพื่อเปิดหลายแหล่งข้อมูล

2

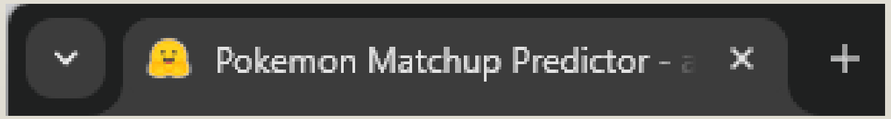
เพื่อช่วยให้ผู้เล่นตัดสินใจได้เร็วว่าจะเลือกใช้โปเกม่อนตัวไหนสู้กับโปเกม่อนของคู่ต่อสู้

3

เพื่อเป็นฐานข้อมูลเชิงตัวเลขไว้ให้ผู้เล่นถกเถียง/อ้างอิงอย่างมีเหตุผลเวลาเปรียบเทียบโปเกม่อน



10 tabs open



1 TAB OPEN!

Less Alt-Tab

although you may still need to check abit of info

ผู้ใช้ของระบบ





ผู้เล่นเกมโปเกม่อนที่อาจจะไม่ชำนาญในการ
เลือกโปเกม่อนที่เหมาะสมในการต่อสู้

ที่มาของข้อมูล

เว็บไซต์: **Kaggle**
https://www.kaggle.com/datasets/lmno3418/pokedex-pokemon-data

ผู้เผยแพร่: **LMNO3418** 
https://www.kaggle.com/lmno3418

ภาพรวม: ข้อมูลนี้จะมีไฟล์ csv อยู่ 2 ไฟล์ ที่เรานำมาใช้พัฒนาระบบ คือ ‘final_pokemon.csv’ และ ‘final_combats.csv’ ซึ่งครอบคลุมแค่โปเกม่อนจาก **เจเนเรชั่น 1-6 เท่านั้น**

final_combats.csv

```
First_pokemon,Second_pokemon,Winner
266,298,298
702,701,701
191,668,668
237,683,683
151,231,151
192,134,134
220,763,763
302,31,31
442,130,130
701,624,701
```

final_pokemon.csv

```
#,Name,Type 1,Type 2,HP,Attack,Defense,Sp. Atk,Sp. Def,Spe
1,Bulbasaur,Grass,Poison,45,49,49,65,65,45,1,False,7,69,64
2,Ivysaur,Grass,Poison,60,62,63,80,80,60,1,False,10,130,14
3,Venusaur,Grass,Poison,80,82,83,100,100,80,1,False,20,100
5,Charmander,Fire,Normal,39,52,43,60,50,65,1,False,6,85,62
6,Charmeleon,Fire,Normal,58,64,58,80,65,80,1,False,11,190,
7,Charizard,Fire,Flying,78,84,78,109,85,100,1,False,17,905
10,Squirtle,Water,Normal,44,48,65,50,64,43,1,False,5,90,63
11,Wartortle,Water,Normal,59,63,80,65,80,58,1,False,10,225
12,Blastoise,Water,Normal,79,83,100,85,105,78,1,False,16,8
14,Caterpie,Bug,Normal,45,30,35,20,20,45,1,False,3,29,39,"
15,Metapod,Bug,Normal,50,20,55,25,25,30,1,False,7,99,72,"{
16,Butterfree,Bug,Flying,60,45,50,90,80,70,1,False,11,320,
17,Weedle,Bug,Poison,40,35,30,20,20,50,1,False,3,32,39,"{"
18,Kakuna,Bug,Poison,45,25,50,25,25,35,1,False,6,100,72,"{
19,Beedrill,Bug,Poison,65,90,40,45,80,75,1,False,10,295,17
21,Pidgey,Normal,Flying,40,45,40,35,35,56,1,False,3,18,50,
22,Pidgeotto,Normal,Flying,63,60,55,50,50,71,1,False,11,30
```

รายละเอียดของข้อมูล

final_pokemon.csv: ข้อมูลพื้นฐานของโปเกม่อนจำนวน 697 ตัว

- #: Pokédex number
- Name: name of the Pokémon
- Type 1: primary type (e.g., Fire, Water)
- Type 2: secondary type (can be empty)
- Hp: hit points (health)
- Attack: physical attack strength
- Defense: physical defense ability
- Special attack: special attack strength
- Special defense: special defense ability
- Speed: determines move order in battle
- Generation: game generation Pokémon belongs to
- Legendary: Whether the Pokémon is legendary (True/False)
- Height: Height of the Pokémon (in decimetres)
- Weight: Weight of the Pokémon (in hectograms)
- Base_experience: Base experience points earned
- Sprites: URL to Pokémon's sprite image

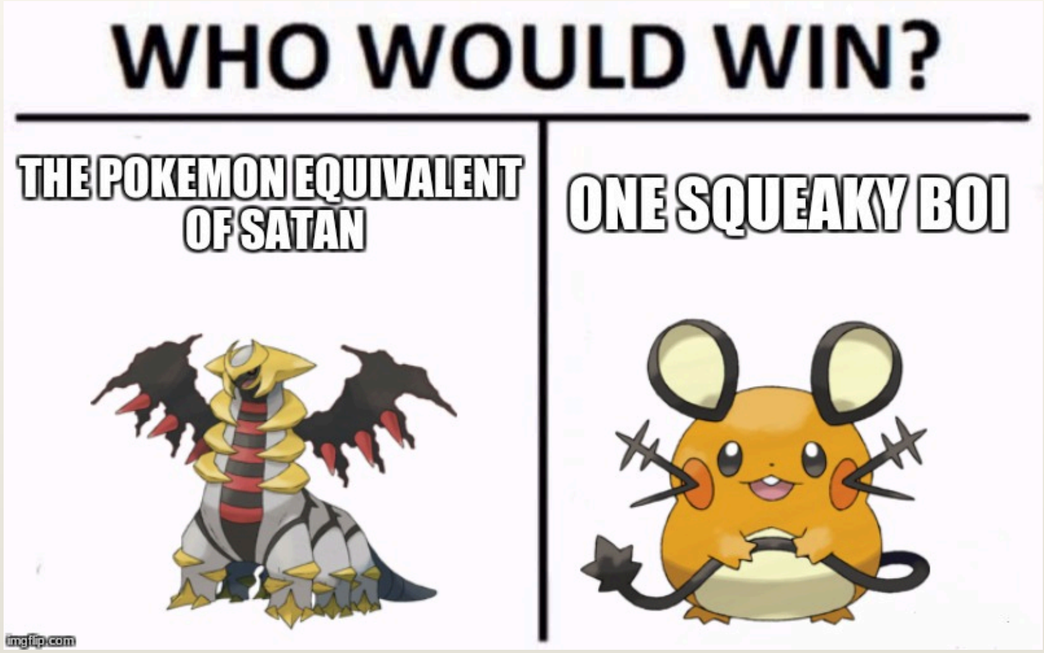
final_combats.csv: ข้อมูลผลการจำลองการต่อสู้ของโปเกม่อนจำนวน 38,054 ครั้ง

- First_pokemon: ID of the first Pokémon in the battle
- Second_pokemon: ID of the second Pokémon in the battle
- Winner: ID of the Pokémon that won the battle



รายละเอียดของสิ่งที่ต้องการทำนาย





สิ่งที่ต้องการทำนายคือผลการต่อสู้ของโปเกม่อนโดยจะแสดงชื่อของโปเกม่อนที่ชนะและโอกาสชนะของโปเกม่อนตัวแรก โดยใช้แค่ stats และ types ของโปเกม่อนในการทำนาย ไม่ได้รวมถึง abilities, items, EV/IV, moves ของโปเกม่อน

ขั้นตอนการเตรียมข้อมูล

Feature Selection

ลบฟีเจอร์ที่ไม่จำเป็น ได้แก่ ส่วนสูง น้ำหนัก และ base exp ของโปเกม่อน เนื่องจากเราคาดการณ์จาก **“ค่าสเตตัส + ธาตุ”** เป็นหลัก ขณะที่ ส่วนสูง/น้ำหนัก เป็นข้อมูลเชิงกายภาพ และ base_experience คือค่าที่สะท้อนการเติบโตของโปเกม่อน

Data Validation & Cleaning

- ตรวจสอบความถูกต้องของข้อมูล เพราะอาจมีข้อมูลที่สะกดผิดหรือไม่มีจริง จึงแก้ไขด้วยการกำหนด Valid Types
- เนื่องจากข้อมูลต้นแบบโปเกม่อนที่มีเพียง 1 ธาตุ จะมีธาตุที่ 2 เป็น Normal ซึ่งไม่ถูกต้อง จึงใช้การแทนที่ Normal ด้วย NA



Feature Duplication: เพื่อแยกข้อมูลโปเกม่อนตัวที่ 1 และ 2 โดย

```
pokemon1 = pokemon_df.add_suffix('_first')
pokemon2 = pokemon_df.add_suffix('_second')
```

Data Integration: รวม dataframe ข้างต้นเป็น ‘merged_df’ โดย

```
merged_df_temp = combats_df.merge(pokemon1, left_on='First_pokemon', right_on='#_first')
merged_df = merged_df_temp.merge(pokemon2, left_on='Second_pokemon', right_on='#_second')
```

สุดท้ายคือ **Labeling** และลบ Feature ที่อาจก่อให้เกิด Overfitting และ Data Leakage

```
merged_df['label'] = (merged_df['Winner'] == merged_df['First_pokemon']).astype('int64')
train_df = merged_df.drop(columns=["Winner", "#_first", "#_second"])
```

ขั้นตอนการเตรียมคุณลักษณะ

Creating New Features from Existing Data

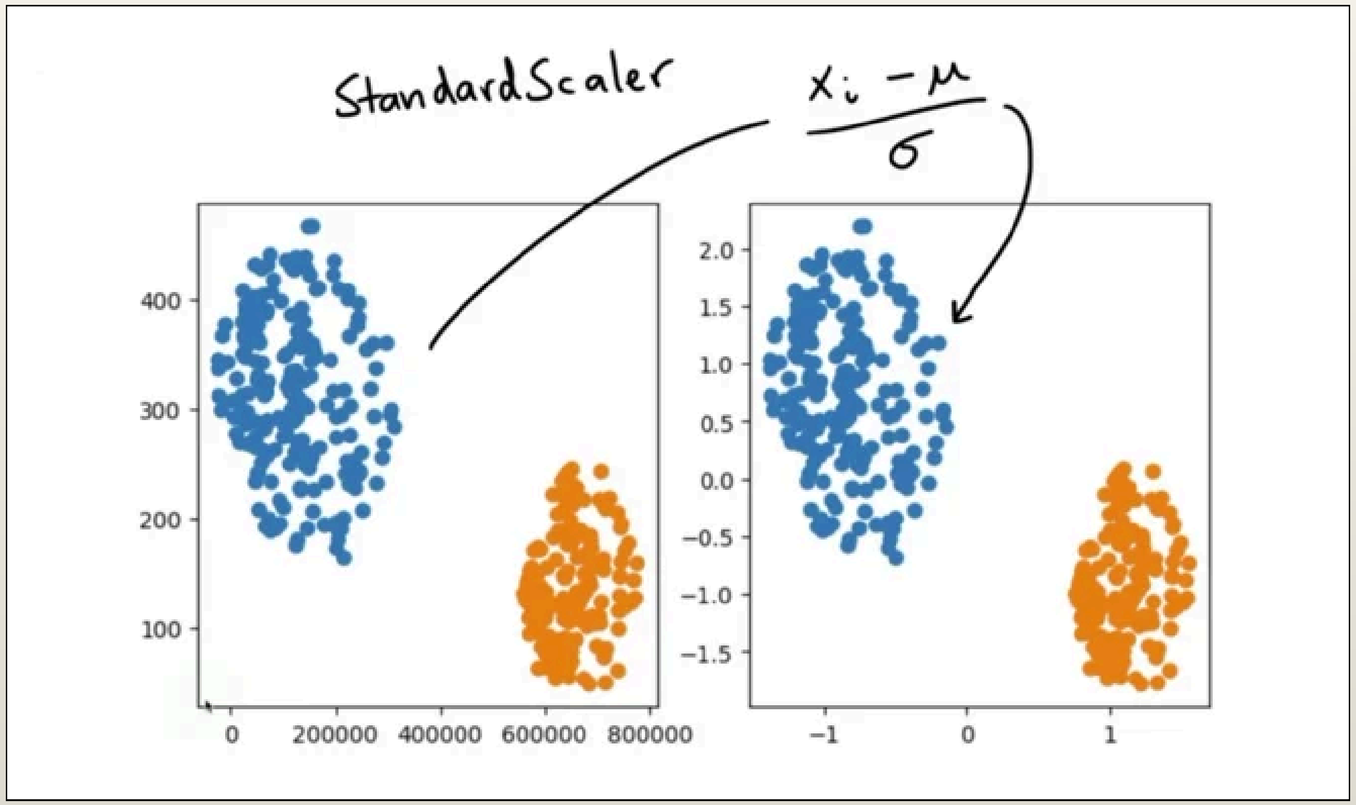
เพิ่มฟีเจอร์ใหม่

- **Stat diffs:** ความต่างของค่าสเตตัสทุกค่าระหว่างโปเกม่อนตัวที่ 1 และ 2 (HP_diff, Attack_diff, Defense_diff, Sp. Atk_diff, Sp. Def_diff, Speed_diff)
- **BST diff:** ผลรวมความแตกต่างของทุกสเตตัส
- **Legendary diff:** ลด bias ว่าโปเกม่อนตัวแรกและตัวที่ 2 เป็น Legendary หรือไม่
- **Type effectiveness:** คำนวณความแรงการโจมตีของฝ่ายหนึ่งต่ออีกฝ่าย (eff_first_on_second, eff_second_on_first)
- **Net type edge:** net_type_edge = eff_first_on_second - eff_second_on_first

โดยพวกเรา**เลือกใช้แค่ฟีเจอร์เหล่านี้สำหรับการเทรนโมเดล**

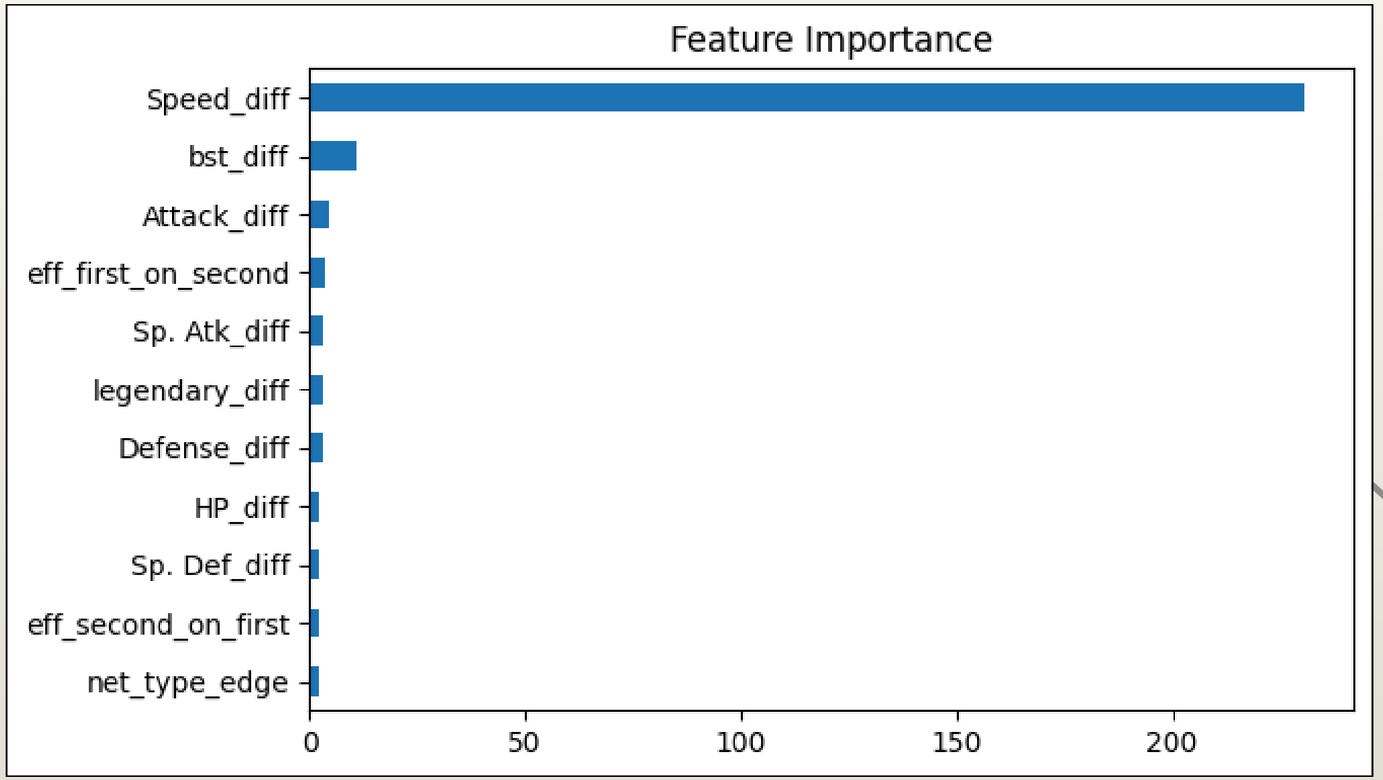
Feature Scaling

เลือกใช้ Standard Scaler กับฟีเจอร์ที่เกี่ยวข้องกับสเตตัสของโปเกม่อน



Feature Importance

จากกราฟจะเห็นได้ว่าฟีเจอร์ที่มีความสำคัญเกินกว่าฟีเจอร์อื่น ๆ อย่างมาก (💀) คือ **‘Speed_diff’** ในขณะที่ฟีเจอร์สเตตัสอื่น ๆ มีความสำคัญอยู่ในระดับที่ใกล้เคียงกัน



Feature Importance

พวกเราจึงเลือกที่จะแก้ไขในขั้นตอน Make Sample ด้วยการนำ Speed_Diff * 0.1 เพื่อลดความสำคัญของฟีเจอร์นั้นลง

```python
def make_sample(a, b):
    a2 = str(a["Type 2"]) if pd.notna(a.get("Type 2")) else "None"
    b2 = str(b["Type 2"]) if pd.notna(b.get("Type 2")) else "None"

    diffs = {
        "HP_diff":        float(a["HP"])        - float(b["HP"]),
        "Attack_diff":    float(a["Attack"])    - float(b["Attack"]),
        "Defense_diff":   float(a["Defense"])   - float(b["Defense"]),
        "Sp. Atk_diff":   float(a["Sp. Atk"])   - float(b["Sp. Atk"]),
        "Sp. Def_diff":   float(a["Sp. Def"])   - float(b["Sp. Def"]),
        "Speed_diff":     0.1 * (float(a["Speed"]) - float(b["Speed"]))
    }
```

Feature Importance

Before



(Pidgeot > Charizard? That's a war crime in Kanto 😡)

After



(Order returns to Kanto; makes sense, as it always should)

ขั้นตอนการเตรียมอัลกอริทึม

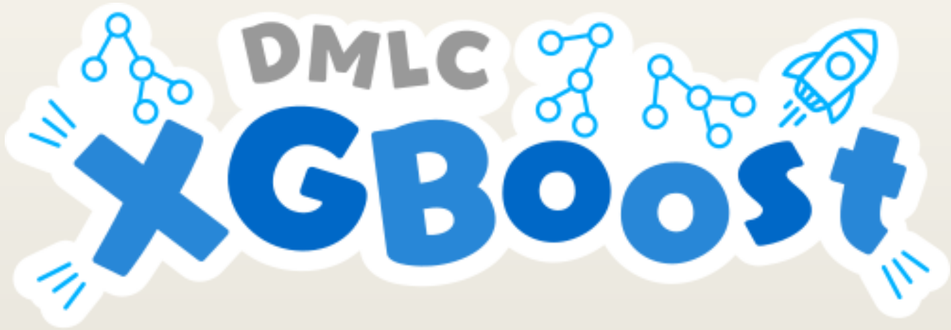
Algorithm Selection

พวกเราได้ทดลองใช้อัลกอริทึม 2 ตัว คือ
Random Forests และ **XGBoost**





Algorithm Selection



จากการทดลอง แม้ **XGBoost** และ **Random Forest** จะให้ค่า Accuracy ใกล้เคียงกัน (~95–99%) แต่เหตุผลที่เราเลือก **XGBoost** เพราะว่าสามารถปรับแต่งไฮเปอร์พารามิเตอร์ได้ละเอียดและยืดหยุ่นกว่า เช่น การปรับ learning rate (η) ซึ่งการปรับข้างต้น แม้จะทำให้ Accuracy ลดลงเล็กน้อย ทว่า ผลลัพธ์การทำนายของโมเดลในสถานการณ์จริงกลับ**เสถียรและสอดคล้องกับความเป็นจริงมากกว่า**

Hyperparameter Tuning

n_estimators – จำนวนรอบการสร้างต้นไม้

- จำนวน weak learners (ต้นไม้) ยิ่งมากยิ่งเรียนรู้ได้ลึกขึ้น แต่ใช้เวลานานและเสี่ยง overfit
- การปรับแต่ง: ลองช่วง 100–2000

max_depth – ความลึกของต้นไม้

- ควบคุมความซับซ้อนของต้นไม้ต่อรอบ
- การปรับแต่ง: ลองช่วง 3–10

colsample_bytree – สัดส่วนฟีเจอร์ต่อรอบ

- สุ่มฟีเจอร์บางส่วนตอนสร้างแต่ละต้นไม้ ลดการพึ่งฟีเจอร์เดิมซ้ำ ๆ
- การปรับแต่ง: ลองช่วง 0.5–1.0

learning_rate (η) – ความเร็วในการเรียนรู้

- ขนาดก้าวของการอัปเดตแต่ละรอบ ยิ่งมากยิ่งเรียนรู้ได้เร็ว เสี่ยงที่จะ overfit
- การปรับแต่ง: ลองช่วง 0.001–1.0

subsample – สัดส่วนตัวอย่างต่อรอบ

- สุ่มตัวอย่างบางส่วนในแต่ละรอบเพื่อลดความแปรปรวน/overfit

การปรับแต่ง: ลองช่วง 0.1–1.0

eval_metric – ตัวชี้วัดระหว่างเทรน

- กำหนด metric ที่ใช้ early stopping/เลือกโมเดล เช่น "logloss", "auc", "error"
- การปรับแต่ง: เลือกให้ตรงกับเป้าหมายงาน (เช่น classification ที่สนใจความน่าจะเป็น = logloss)

ผลการประเมินโมเดล

EVALUATION

Accuracy: 0.94

Cross-validation (Mean of 5) : 0.942

BASELINE

Uniform : 0.494

Stratified: 0.503

Most Frequent (Zero Rule): 0.529

```python
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import cross_val_score

dummy_uniform = DummyClassifier(strategy="uniform", random_state=123)
dummy_mostfreq = DummyClassifier(strategy="most_frequent", random_state=123)
dummy_stratified = DummyClassifier(strategy="stratified", random_state=123)

dummy_uniform.fit(pokemon_train, pokemon_train[label])
dummy_mostfreq.fit(pokemon_train, pokemon_train[label])
dummy_stratified.fit(pokemon_train, pokemon_train[label])

print("Uniform:", dummy_uniform.score(pokemon_val, pokemon_val[label]))
print("Most Frequent:", dummy_mostfreq.score(pokemon_val, pokemon_val[label]))
print("Stratified:", dummy_stratified.score(pokemon_val, pokemon_val[label]))

cv_score = cross_val_score(pokemon_pipeline, pokemon_train[NUM_COLS], pokemon_train[label])
print("Cross-validation mean:", cv_score.mean())
```

✓ 5.1s

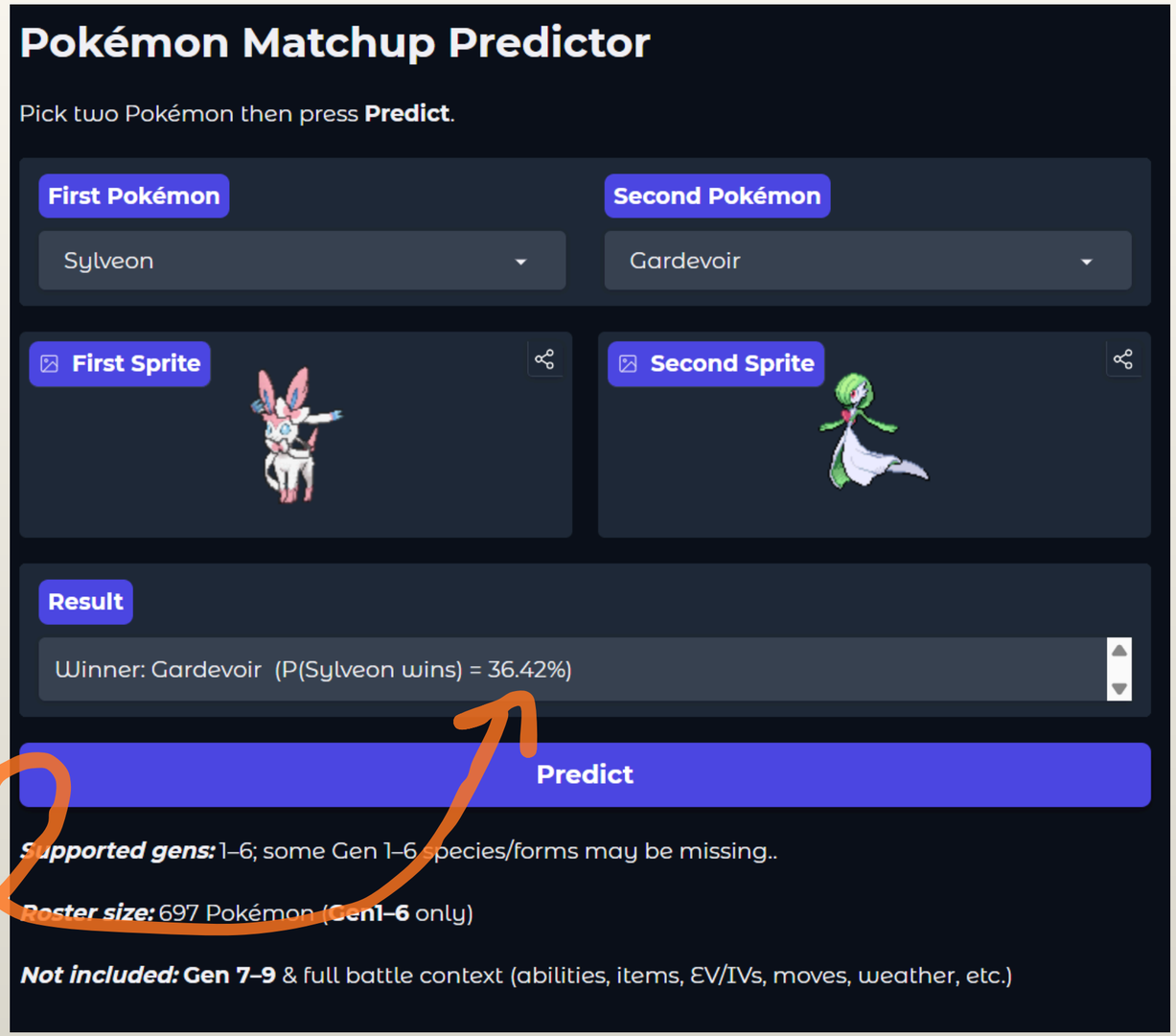
```
Uniform: 0.4940867279894875
Most Frequent: 0.5293035479632063
Stratified: 0.5038107752956636
Cross-validation mean: 0.9426798373405092
```

วิธีการใช้งานระบบ

1. เลือกโปเกมอนทั้งสองตัวที่จะมาต่อสู้กันโดยตัวที่เลือก**ห้าม**ซ้ำกัน

2. กดปุ่ม **Predict** เพื่อดูผลลัพธ์ของการต่อสู้

*โอกาสชนะของโปเกมอนตัวแรก



การมีส่วนร่วมของสมาชิกในกลุ่ม

นายวีรวิชญ์ นิธิคไพศาลกุล รหัสนิสิต 6710451313 หมู่ 200

- Data Analysis
- Model Development
- Model Deployment

นายณัฐพงศ์ คำสียา รหัสนิสิต 6710450813 หมู่ 200

- Model Evaluation
- Model Testing

การเปิดเผยการใช้เครื่องมือปัญญาประดิษฐ์



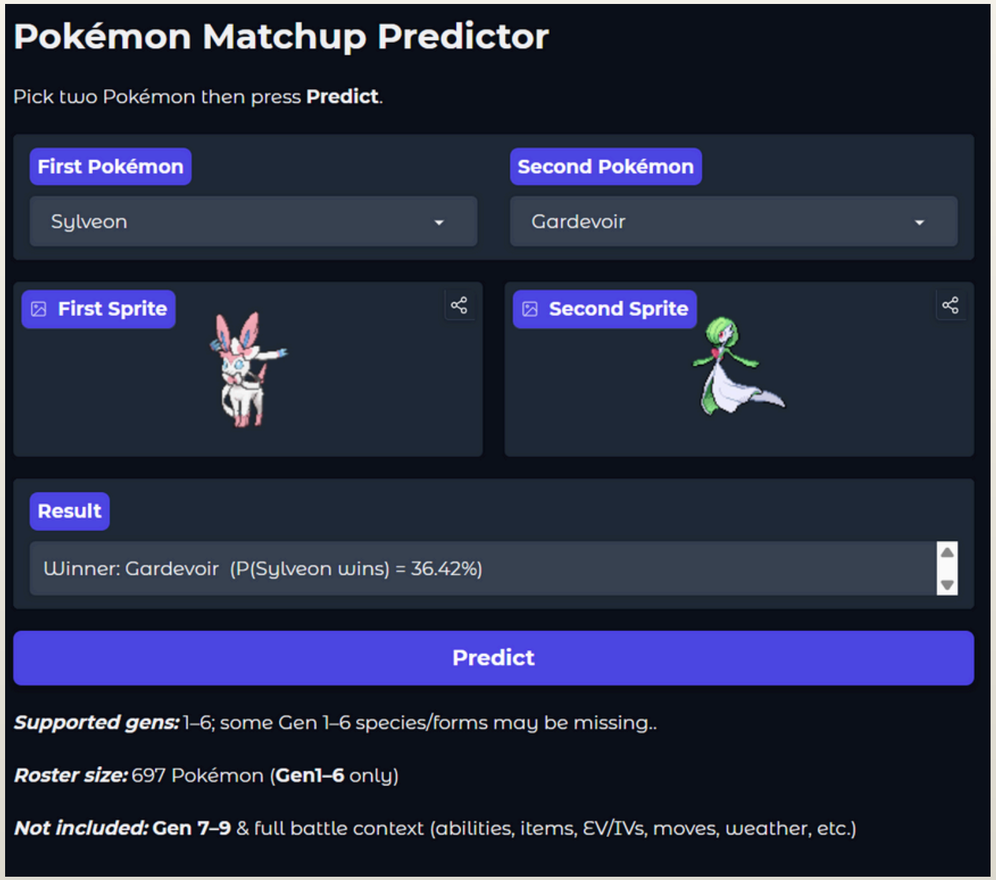
GPT-5

- ใช้เพื่อตกแต่ง Gradio เช่น การเขียนโค้ดดึง URL Sprite ของโปเกม่อนจาก csv มาแสดง
- ใช้เพื่อหาวิธีคำนวณ Type Effectiveness
- ใช้เพื่อแปลและตรวจสอบข้อความภาษาอังกฤษ

ลิงค์ไปยัง Hugging Face Spaces

Click Here

Scan Here



Thank You

